

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607

Documento de Especificação de Projeto Integrador

REVIVE

Sumário

1. PARTE 00 - HISTÓRICO DE VERSÃO DO PROJETO	3
2. PARTE 01 – IDENTIFICAÇÃO DO PRODUTO, EQUIPE E REPOSITÓRIO	3
3. 1 IDENTIFICAÇÃO DO PRODUTO	3
4. 2 DESCRIÇÃO GERAL DO PRODUTO	3
5. 2.1 Resumo do Negócio	3
6. 2.2 Problemas Identificados	4
7. 3 EQUIPE DO PROJETO	4
8. 4 RESPOSITÓRIOS E ARTEFATOS DO PROJETO	5
9. PARTE 02 – PLANEJAMENTO DO SEMESTRE	6
10. 5 SITUAÇÃO ATUAL DO PROJETO	6
11. 6 OBJETIVOS DO SEMESTRE	7
12. 7 CRONOGRAMA DO SEMESTRE	7
13. PARTE 03 – ENTENDIMENTO DO PÚBLICO, DO NEGÓCIO E LEVANTAMENTO INICIAL DE REQUISITOS	8
14. 8 PÚBLICO -ALVO DO PRODUTO	8
15. 9 CONTEXTO DE NEGÓCIO	9
16. 10 SOLUÇÕES EXISTENTES	9
17. 11 PESQUISA COM USUÁRIOS	10
18. 12 RESULTADOS DE PESQUISA COM USUÁRIO	10
19. 13 REGRAS DE NEGÓCIO	10

20.	14	REQUISITOS FUNCIONAIS	11
21.	15	REQUISITOS NÃO FUNCIONAIS	11
22.	PARTE 04 – DESIGN, MODELAGEM, DADOS E SOLUÇÃO TECNOLÓGICA		11
23.	16	DESIGN DA SOLUÇÃO E EXPERIÊNCIA DO USUÁRIO	11
24.	16.1	Fluxo de Navegação	11
25.	16.2	Protótipos do Sistema	12
26.	17	GESTÃO DO PROJETO DE MODELAGEM DO SISTEMA	12
27.	17.1	Organização do Projeto (Abordagem Ágil)	12
28.	17.2	Modelagem e Diagramas do Sistema	12
29.	18	DADOS E MODELAGEM DA INFORMAÇÃO	13
30.	18.1	Modelo de Dados	13
31.	18.2	Estrutura e Manipulação de Dados	13
32.	19	SOLUÇÃO TECNOLÓGICA E EXPERIMENTAÇÃO TÉCNICA INICIAL	13
33.	19.1	Escolhas Tecnológicas	14
34.	19.2	Experimentação Técnica e Primeiros Testes	14
35.	PARTE 05 – ARQUITETURA, TECNOLOGIA E IMPLEMENTAÇÃO EVOLUTIVA DO SISTEMA		14
36.	20	ARQUITETURA GERAL DO SISTEMA	14
37.	21	TECNOLOGIAS E FERRAMENTAS DO PROJETO	15
38.	22	IMPLEMENTAÇÃO POR CAMADAS DO SISTEMA	15
39.	22.1	Frontend	16
40.	22.2	Backend	16
41.	23	ORGANIZAÇÃO DO CÓDIGO E REPOSITÓRIO	17
42.	PARTE 06 – QUALIDADE, SEGURANÇA, AVALIAÇÃO E EXTENSÕES DO PROJETO		17
43.	24	QUALIDADE DO SOFTWARE E TESTES	17
44.	25	SEGURANÇA DA INFORMAÇÃO	18
45.	26	EXTENSÕES E TECNOLOGIAS COMPLEMENTARES	18

46. 27 AVALIAÇÃO GERAL DO PROJETO INTEGRADOR 19

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607

PARTE 00 - HISTÓRICO DE VERSÃO DO PROJETO

| Versão | Período | Fase | Data | Descrição |

|---|---|---|---|---|

| 1.0 | 1º | 01 | 08/03/2026 | Criação inicial do documento do projeto, definição do nome do produto, descrição geral, identificação da equipe e planejamento do semestre. |

| 1.1 | 1º | 02 | 19/04/2026 | Ajustes na descrição do produto, inclusão dos objetivos do semestre, revisão do planejamento e primeiros refinamentos a partir do feedback recebido. |

| 1.2 | 1º | 02 | 19/04/2026 | Revisão das seções de planejamento, arquitetura, implementação por camadas, qualidade de software e segurança da informação. Nesta versão, o documento foi comparado com o código real do projeto, os testes automatizados foram executados localmente e foram registrados pontos de melhoria identificados na navegação, na cobertura de testes e nos controles de segurança. |

PARTE 01 – IDENTIFICAÇÃO DO PRODUTO, EQUIPE E REPOSITÓRIO

1 IDENTIFICAÇÃO DO PRODUTO

Nome do Produto: REVIVE

Descrição do Produto:

O REVIVE é um aplicativo web voltado para pessoas que desejam acompanhar e superar vícios e

dependências, oferecendo ferramentas de registro diário, acompanhamento de abstinência e

suporte motivacional. O produto permite que o usuário cadastre os vícios que deseja combater,

registre diariamente seu humor, gatilhos e conquistas, acompanhe o tempo de abstinência em

dias e visualize a economia financeira gerada ao deixar de gastar com o vício. Além disso, o REVIVE

oferece um sistema de metas pessoais, um histórico de recaídas com espaço para reflexão, um

calendário de atividades, conquistas desbloqueáveis e mensagens motivacionais diárias. O produto foi pensado para ser um companheiro de recuperação acessível, organizado e encorajador, auxiliando o usuário a manter o controle sobre sua jornada de superação de forma prática e visual.

2 DESCRIÇÃO GERAL DO PRODUTO

O REVIVE está inserido no contexto de saúde e bem-estar pessoal, mais especificamente na área

de recuperação de dependências e mudança de hábitos. Milhões de pessoas lidam diariamente

com vícios que afetam sua saúde física, mental e financeira, como tabagismo, alcoolismo, jogos de

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 azar, uso excessivo de redes sociais, entre outros. A atividade principal desse contexto é o

acompanhamento contínuo do progresso de abstinência, o registro de experiências diárias e a

busca por motivação para manter-se firme no processo de recuperação. Esse cenário é relevante

porque a falta de ferramentas acessíveis e organizadas para automonitoramento contribui para

altas taxas de recaída e abandono do tratamento, tornando essencial a existência de soluções

digitais que apoiem o usuário de forma prática e contínua.

2.2 Problemas Identificados

Item Descrição

O problema de Ausência de uma ferramenta unificada para registro e acompanhamento da jornada de recuperação de vícios, fazendo com que o usuário não consiga visualizar seu progresso de forma clara e organizada.

Afeta Pessoas em processo de recuperação de vícios e dependências que buscam acompanhar sua evolução de forma autônoma.

Cujo impacto é Falta de consciência sobre o próprio progresso, dificuldade em identificar padrões de gatilhos e recaídas, desmotivação pela ausência de reconhecimento das conquistas e perda de controle sobre a economia financeira gerada pela abstinência.

Benefícios de uma solução seriam Maior controle e consciência sobre a jornada de recuperação, identificação de padrões comportamentais, motivação contínua por meio de conquistas e métricas visuais, e clareza sobre o impacto financeiro positivo da abstinência.

3 EQUIPE DO PROJETO

Matrícula Nome Completo Função no Projeto

2310796 Vitor de Aguiar Adrião Product Owner

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 2310685 João Marcelo Pin to Desenvolvedor Back -end

2311143 Davi Maestri Ribeiro Desenvolvedor Front -end

Renato Luan Orientador Principal

Orientador Secundário

4 RESPOSITÓRIOS E ARTEFATOS DO PROJETO

Tipo de

Artefato Link

Descrição

Repositório

de Código AINDA NÃO FOI CRIADO UM REPOSITORIO Repositório principal

contendo o
código -fonte do
backend (API
Express) e do
frontend
(React).
Documentação
da API <http://localhost:3000/api/docs> (Swagger UI) Documentação
interativa da
API REST gerada
automaticamen
te via
Swagger/Open
API.
Banco de
Dados Supabase (PostgreSQL em nuvem) Banco de dados
hospedado no
Supabase,
contendo as
tabelas do
sistema REVIVE.
Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515
Secretaria Setorial – Bloco I
Fone: (62) 3310 6658 | FAX: (62) 3310 6607 Protótipo /
Aplicação
Web (inserir link se houver deploy) Aplicação web
frontend

desenvolvida

em React,

servida pelo

Vite em

ambiente de

desenvolviment

o.

Notion

(Semestre

Anterior) [https://www.notion.so/REVIVE -](https://www.notion.so/REVIVE-1e3adcf9a7868044aaebf445e2247305)

1e3adcf9a7868044aaebf445e2247305 Workspace do

projeto com

documentação

e artefatos do

semestre

anterior.

Figma

(Protótipo

Visual) <https://www.figma.com/design/1keY3WNeLoTr4KGegWctv>

V/REVIVE -WEB -v.1.2 Protótipo visual

e design da

aplicação web.

PARTE 02 - PLANEJAMENTO DO SEMESTRE

5 SITUAÇÃO ATUAL DO PROJETO

O projeto REVIVE encontra-se em uma fase funcional de desenvolvimento, com backend, frontend e banco de dados já integrados. A aplicação atual é uma plataforma web para acompanhamento da jornada de recuperação de vícios, permitindo cadastro e login de usuário, registro de vícios, acompanhamento de dias de abstinência, cálculo de economia

financeira, registro diário de humor e observações, histórico de recaídas, metas pessoais, conquistas, relatórios e visualização de informações em dashboard.

O backend foi implementado em Node.js com Express.js e concentra as rotas da API REST no arquivo `index.js`. A API utiliza autenticação com JWT, armazenamento de senhas com bcrypt, limitação de requisições com `express-rate-limit`, configuração de CORS, logs com Morgan e documentação Swagger/OpenAPI em `/api/docs`. As rotas principais cobrem autenticação, perfil do usuário, vícios, registros diários, recaídas, metas e mensagens motivacionais. O acesso ao banco é feito por meio da biblioteca `@supabase/supabase-js`.

O banco de dados está estruturado no Supabase/PostgreSQL. O schema informado contém as tabelas `usuarios`, `vicios`, `registros_diarios`, `historico_recaidas`, `metas`, `mensagens_motivacionais` e `marcos`. No código atual, as seis primeiras tabelas são utilizadas diretamente pela aplicação. A tabela `marcos` existe no schema, mas não foi encontrada integração direta dela nas rotas ou telas atuais, portanto deve ser tratada como possibilidade de evolução e não como funcionalidade já implementada.

O frontend foi desenvolvido com React, Vite e Tailwind CSS. A aplicação possui páginas de Login, Dashboard, Analytics, Detalhes, Metas, Calendário, Conquistas, Relatórios, Perfil e Dicas. A organização segue uma separação por páginas, componentes, contextos, serviços, utilitários e configuração de ambiente. O estado principal fica distribuído entre `AuthContext`, `DataContext` e `UIContext`, enquanto a comunicação com a API é encapsulada nos arquivos da pasta `services`.

Durante a revisão da Fase 02, foram identificados pontos que precisam de ajuste. O botão “Novo” da barra de navegação aponta para `/novo-vicio`, mas essa rota não existe no roteamento atual; na prática, o cadastro de novo vício acontece pelo wizard exibido no Dashboard. Também foi encontrada uma diferença de nomenclatura na barra de navegação: o componente tenta ler `vicioSelecionado`, enquanto o contexto de dados expõe `selectedAddiction`. Isso não compromete o funcionamento central do sistema, mas impede que o item dinâmico de detalhes funcione como descrito nos comentários do código.

O projeto já possui testes automatizados com Vitest, Supertest e React Testing Library. A validação local executada com `npm run validate` passou, incluindo testes da API, testes do painel e build de produção do frontend. Apesar disso, a cobertura ainda é limitada e se concentra em funções auxiliares, validações iniciais e alguns comportamentos de interface. Fluxos críticos, como cadastro completo, login real com banco, criação de vício, recaída, metas e autorização entre usuários, ainda precisam de testes mais abrangentes.

6 OBJETIVOS DO SEMESTRE

Para esta fase, o foco do projeto deixa de ser apenas implementar telas e rotas e passa a incluir estabilização, validação e análise crítica do que já foi construído. Os objetivos do semestre são:

1. consolidar as funcionalidades principais já implementadas, principalmente autenticação, gerenciamento de vícios, registros diários, recaídas, metas e relatórios;

2. corrigir inconsistências encontradas durante a revisão, como a rota inexistente ``/novo-vicio`` e a diferença de nomenclatura entre a navegação e o contexto de dados;
3. ampliar os testes automatizados para cobrir fluxos completos da API e da interface, principalmente cadastro, login, criação de vício, registro diário, recaída e metas;
4. registrar testes manuais de navegação e uso do sistema, pois atualmente o repositório tem evidência automatizada, mas não possui roteiro formal de testes manuais;
5. revisar a segurança da informação, com atenção ao uso de JWT em ``localStorage``, validação de entrada, isolamento dos dados por usuário e configuração real do Supabase;
6. documentar a arquitetura real do sistema com base no código, evitando tratar comentários antigos ou afirmações do documento como verdade sem validação;
7. preparar o projeto para uma entrega mais madura, com ``env.example``, documentação de execução, evidências de teste e, se possível, scripts ou migrations do banco.

Esses objetivos foram definidos porque o sistema já possui uma base funcional, mas ainda precisa de ajustes de qualidade para reduzir risco de falhas em fluxos importantes. A revisão mostrou que a aplicação não está apenas em fase de protótipo visual, porém também não deve ser descrita como finalizada ou totalmente validada.

7 CRONOGRAMA DO SEMESTRE

O cronograma do semestre pode ser organizado em seis etapas:

****Análise e diagnóstico (Semanas 1-2):**** revisão do documento, conferência do código real, identificação das funcionalidades implementadas, verificação da estrutura do banco e levantamento de inconsistências entre documentação e projeto.

****Correções funcionais (Semanas 3-5):**** ajuste da navegação para criação de novo vício, correção da nomenclatura usada entre ``NavBar`` e ``DataContext``, revisão de mensagens de erro e melhoria de pequenos comportamentos da interface.

****Testes e validação (Semanas 6-8):**** ampliação dos testes automatizados da API e do frontend, criação de roteiros de testes manuais e execução dos fluxos principais: cadastro, login, criação de vício, registro diário, recaída, metas, relatórios e exportação.

****Segurança e banco de dados (Semanas 9-10):**** revisão das permissões no Supabase, documentação das políticas de acesso, avaliação do armazenamento do token, validação dos dados recebidos pela API e criação de arquivo ``env.example`` sem chaves reais.

****Documentação e preparação de entrega (Semanas 11-13):**** atualização do documento de especificação, revisão da documentação Swagger, melhoria do README, organização das evidências de teste e preparação do build do frontend.

****Validação final (Semanas 14-16):**** execução completa da validação local, testes finais de uso, correção de problemas encontrados na entrega, revisão da apresentação e fechamento da documentação acadêmica.

PARTE 03 – ENTENDIMENTO DO PÚBLICO, DO NEGÓCIO E

LEVANTAMENTO INICIAL DE REQUISITOS

8 PÚBLICO -ALVO DO PRODUTO

O público -alvo do REVIVE é composto por pessoas que desejam superar vícios ou dependências e

buscam uma ferramenta digital para acompanhar sua jornada de recuperação de forma autônoma.

Esse público inclui desde jovens adultos até pessoas de meia -idade que lidam com vícios como

tabagismo, alcoolismo, dependência de jogos de azar, uso excessivo de redes sociais, compulsão

alimentar, entre outros.

O perfil principal do usuário é alguém que possui familiaridade básica com tecnologia e smartphones, que busca organização e motivação para manter a abstinência, e que valoriza a

privacidade no acompanhamento de sua recuperação. Existem dois perfis de uso principais: o

usuário que está iniciando o processo de recuperação e precisa de suporte motivacional constante,

e o usuário que já está em estágio mais avançado de abstinência e deseja monitorar seu progresso,

identificar padrões e manter -se vigilante contra recaídas.

O REVIVE não substitui acompanhamento profissional de saúde, mas atua como uma ferramenta

complementar que oferece registro estruturado, visualização de progresso e incentivo contínuo,

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 podendo ser utilizado de forma independente ou em conjunto com tratamentos médicos e

psicológicos.

9 CONTEXTO DE NEGÓCIO

O REVIVE está inserido no mercado de aplicativos de saúde e bem-estar digital, uma área em

crescimento acelerado impulsionada pela maior conscientização sobre saúde mental e pela popularização do uso de smartphones como ferramentas de autocuidado. Especificamente, o

produto se posiciona no segmento de aplicativos de recuperação de dependências e mudança de

hábitos.

O contexto em que o problema ocorre é o do dia a dia de pessoas que enfrentam vícios e precisam

de acompanhamento contínuo, muitas vezes sem acesso frequente a profissionais de saúde ou

grupos de apoio presenciais. A atividade principal nesse cenário é o monitoramento da

abstinência, o registro de experiências diárias e a identificação de gatilhos que podem levar a

recaídas.

As dificuldades existentes incluem a falta de ferramentas específicas e integradas para esse

acompanhamento, a dependência de anotações manuais desorganizadas, a ausência de métricas

claras de progresso e a carência de estímulos motivacionais no momento certo. Essas lacunas

representam oportunidades de melhoria que o REVIVE busca preencher, oferecendo uma solução

digital organizada, visual e encorajadora que centraliza todas as informações da jornada de recuperação em um único lugar.

10 SOLUÇÕES EXISTENTES

No mercado atual, existem algumas soluções voltadas para o acompanhamento de vícios e mudança

de hábitos. Aplicativos como "I Am Sober" e "Quit Tracker" oferecem contadores de abstinência e

notificações motivacionais, porém geralmente focam em um único tipo de vício (como álcool ou

tabaco) e possuem funcionalidades limitadas no plano gratuito. Outras soluções, como "HabitBull"

e "Loop Habit Tracker", são voltadas para o rastreamento genérico de hábitos, sem funcionalidades

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 específicas para recuperação de dependências, como registro de recaídas com reflexão ou

acompanhamento financeiro da economia gerada pela abstinência.

Processos manuais também são comuns: muitos usuários utilizam planilhas, cadernos ou aplicativos

de notas para registrar seu progresso, o que resulta em informações desorganizadas e difíceis de

visualizar ao longo do tempo. Grupos de apoio presenciais, embora valiosos, não oferecem uma

ferramenta digital integrada para monitoramento individual contínuo.

O REVIVE se diferencia dessas soluções por oferecer, em uma única plataforma, o acompanhamento

simultâneo de múltiplos vícios, cálculo automático da economia financeira, registro diário com

humor e gatilhos, sistema de metas, histórico de recaídas com espaço para reflexão, conquistas

desbloqueáveis e mensagens motivacionais, tudo em uma interface moderna e responsiva.

11 PESQUISA COM USUÁRIOS

No semestre anterior, foi definido um Pipeline Funcional para coleta de dados com objetivo de

coletar informações sobre hábitos e comportamentos relacionados a vícios em jovens adultos (18 a

30 anos). A pesquisa abrangeu campos como identificação do participante, informações sobre o

vício principal, fatores emocionais e ambientais, e impactos e percepções.

12 RESULTADOS DE PESQUISA COM USUÁRIO

No semestre anterior, a análise de uma amostra simulada de 20 jovens adultos revelou que o cigarro

é o vício mais prevalente (60%), seguido do álcool (20%). Os principais gatilhos identificados foram

Ansiedade (30%) e Estresse (25%). Esses dados justificam a integração de recursos de bem-estar

emocional e a personalização das mensagens motivacionais diárias no aplicativo.

13 REGRAS DE NEGÓCIO

ID Descrição Requisitos Relacionados

RNE -001 O usuário deve possuir uma conta autenticada para

acessar qualquer funcionalidade do sistema além do RF-001, RF -002, RNF -001

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 login.

Detalhamento: O sistema utiliza autenticação baseada em JWT com validade de 7 dias. Todas as rotas protegidas exigem

um token válido no cabeçalho Authorization.

14 REQUISITOS FUNCIONAIS

ID Descrição Status Prioridade

RF-001 O sistema deve permitir que o usuário se cadastre

com nome, e-mail e senha. Implementado Alta

Detalhamento: O cadastro é realizado pela rota POST /api/auth/cadastro. O sistema valida a unicidade do e-mail, a força

da senha (mínimo 6 caracteres, maiúscula e caractere especial) e armazena a senha criptografada com bcrypt.

15 REQUISITOS NÃO FUNCIONAIS

ID Descrição Status Prioridade

RNF-001 O sistema deve garantir a segurança dos dados do

usuário por meio de criptografia de senhas e

autenticação por token. Implementado Alta

Detalhamento: Senhas são armazenadas com hash bcrypt (10 salt rounds). A autenticação utiliza JWT com expiração

de 7 dias. Tokens são validados em todas as requisições protegidas.

PARTE 04 – DESIGN, MODELAGEM, DADOS E SOLUÇÃO TECNOLÓGICA

16 DESIGN DA SOLUÇÃO E EXPERIÊNCIA DO USUÁRIO

16.1 Fluxo de Navegação

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 O fluxo de navegação do REVIVE é organizado da seguinte forma: o usuário inicia na Tela de

Login/Cadastro, onde pode fazer login ou criar uma nova conta. Após autenticação, é redirecionado ao Dashboard (Painel Principal) com KPIs, heatmap de atividades, cards dos vícios e

timeline de atividades recentes. A partir do Dashboard, o usuário pode acessar: Detalhes do Vício

(estatísticas aprofundadas e histórico), Metas (criação e gerenciamento de metas pessoais),

Analytics (gráficos e tendências), Calendário (visão em calendário dos registros), Conquistas

(badges desbloqueáveis), Relatórios (relatórios imprimíveis), Dicas (orientações sobre recuperação) e Perfil (configurações e alternância de tema). A navegação é realizada por uma barra

lateral (NavBar) persistente, acompanhada de um cabeçalho (Header) com informações do usuário

e botão de logout.

16.2 Protótipos do Sistema

O sistema foi desenvolvido com design glassmorphism, utilizando as fontes Space Grotesk (títulos)

e Manrope (corpo), com paleta baseada em verde esmeralda (#7CF6C4) e azul ciano (#35D3FF)

sobre fundo escuro (#050910). Os componentes utilizam efeitos de vidro (backdrop blur), bordas

sutis com gradiente e animações suaves com Framer Motion. Os protótipos visuais do semestre

anterior estão disponíveis no Figma.

17 GESTÃO DO PROJETO DE MODELAGEM DO SISTEMA

17.1 Organização do Projeto (Abordagem Ágil)

O projeto utiliza Git para controle de versão e está organizado em um repositório monolítico contendo tanto o backend quanto o frontend. O desenvolvimento segue ciclos curtos de implementação, com foco em entregas incrementais de funcionalidades. As convenções de código

estão documentadas em docs/coding-conventions.md, estabelecendo padrões de nomenclatura

e estrutura de código.

17.2 Modelagem e Diagramas do Sistema

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 O sistema REVIVE é modelado em torno de seis entidades principais: Usuário (entidade central,

possuindo vícios e metas), Vício (pertence a um usuário e possui registros diários e histórico de

recaídas), Registro Diário (vinculado a um vício específico), Recaída (vinculada a um vício,

registrando dias de abstinência perdidos), Meta (pertence a um usuário e pode ser vinculada

opcionalmente a um vício) e Mensagem Motivacional (independente, acessada aleatoriamente).

18 DADOS E MODELAGEM DA INFORMAÇÃO

18.1 Modelo de Dados

O banco de dados do REVIVE está hospedado no Supabase (PostgreSQL em nuvem) e é composto

por seis tabelas: usuarios (id, nome, email, senha_hash, created_at, updated_at), vicios (id,

usuario_id, nome_vicio, data_inicio, data_ultima_recaida, valor_economiza do_por_dia, ativo,

data_criacao), registros_diarios (id, vicio_id, data_registro, humor, gatilhos, conquistas, observacoes, created_at), historico_recaidas (id, vicio_id, data_recaida, motivo, dias_abstinencia_perdidos, created_at), metas (id, usuario_id, vicio_id, descricao_meta, dias_objetivo, valor_objetivo, concluida, data_criacao) e mensagens_motivacionais (id, mensagem, tipo_vicio, ativa, created_at).

18.2 Estrutura e Manipulação de Dados

Os dados do REVIVE são armazenados em um banco PostgreSQL hospedado no Supabase,

acessado pelo backend por meio da biblioteca @supabase/supabase-js. A comunicação com o

banco é feita exclusivamente pelo servidor Express, que atua como intermediário entre o frontend

e o banco de dados. A manipulação de dados segue o padrão de operações CRUD para cada

entidade. Os cálculos de estatísticas (dias de abstinência, economia financeira, duração formatada)

são realizados no backend pela função calculateAddictionStatistics() antes de enviar os dados ao

frontend.

19 SOLUÇÃO TECNOLÓGICA E EXPERIMENTAÇÃO TÉCNICA INICIAL

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607

19.1 Escolhas Tecnológicas

As tecnologias do REVIVE foram escolhidas com base na adequação ao problema, na experiência

da equipe e no ecossistema disponível. Node.js com Express.js foi escolhido para o backend por

ser leve, rápido de desenvolver e amplamente utilizado. React 19 foi escolhido para o frontend por

oferecer componentização e reatividade. Vite foi adotado como build tool por sua velocidade.

Tailwind CSS permite estilização rápida e consistente. Supabase (PostgreSQL) oferece banco

gerenciado em nuvem. JWT e bcrypt foram adotados para autenticação e segurança. Framer

Motion para animações e Vitest para testes.

19.2 Experimentação Técnica e Primeiros Testes

As primeiras experimentações técnicas incluíram: conexão com o Supabase e operações CRUD,

implementação e teste do fluxo completo de autenticação JWT, testes de hash de senhas com

bcrypt, criação de componentes React básicos para validar a estrutura de componentização,

implementação de testes unitários para funções auxiliares (sanitize, formatDuration,

calculateAddictionStats) e testes de integração para rotas da API, e experimentação inicial com

Service Worker para suporte offline.

PARTE 05 – ARQUITETURA, TECNOLOGIA E IMPLEMENTAÇÃO EVOLUTIVA

DO SISTEMA

20 ARQUITETURA GERAL DO SISTEMA

O REVIVE utiliza uma arquitetura cliente-servidor em três camadas: apresentação, aplicação e dados. Essa divisão foi confirmada no código e ajuda a separar a interface do usuário, as regras da API e o armazenamento das informações.

A camada de apresentação é uma SPA desenvolvida em React com Vite. Ela é responsável pela navegação, telas, formulários, feedback visual, gráficos, relatórios e organização da experiência do usuário. O frontend não acessa diretamente o Supabase. Toda comunicação externa passa pela camada de serviços em `src/services`, que utiliza `fetch` para chamar a API REST. O token JWT é armazenado no `localStorage` com a chave `revive_token` e enviado no cabeçalho `Authorization: Bearer <token>` nas requisições protegidas.

A camada de aplicação é uma API REST construída com Express.js. Ela centraliza autenticação, autorização, validações iniciais, cálculo de estatísticas e comunicação com o banco. O backend implementa rotas para cadastro, login, perfil, vícios, registros diários, recaídas, metas e mensagens motivacionais. Também possui middleware de autenticação JWT, CORS, rate limiting, logs HTTP com Morgan e documentação Swagger/OpenAPI.

A camada de dados utiliza Supabase com PostgreSQL. O schema informado contém tabelas para usuários, vícios, registros diários, histórico de recaídas, metas, mensagens motivacionais e marcos. A aplicação atual utiliza diretamente as tabelas `usuarios`, `vicios`, `registros\_diarios`, `historico\_recaidas`, `metas` e `mensagens\_motivacionais`. A tabela `marcos` não aparece integrada no código analisado, então não deve ser apresentada como funcionalidade ativa.

O fluxo principal começa quando o usuário acessa o frontend e realiza login ou cadastro. No login, o backend consulta a tabela `usuarios`, compara a senha recebida com o hash bcrypt armazenado e retorna um JWT quando as credenciais são válidas. Depois disso, o frontend usa esse token para carregar vícios, metas, registros, recaídas e mensagens. Ao listar vícios, o backend calcula dias de abstinência, tempo formatado e valor economizado antes de devolver os dados ao frontend.

Depois da revisão de testes e segurança, a arquitetura continua adequada para o porte atual do projeto, mas alguns cuidados precisam ser registrados. A API faz o isolamento dos dados por `usuario\_id` em várias rotas, o que reduz risco de acesso indevido. Porém, o repositório não contém migrations nem políticas RLS do Supabase, então não é possível afirmar pelo código local que o banco está protegido independentemente da API. Também foi observado que a sanitização atual é básica, baseada principalmente em `trim()`, e não deve ser descrita como proteção completa contra XSS ou payloads maliciosos.

21 TECNOLOGIAS E FERRAMENTAS DO PROJETO

As principais tecnologias utilizadas no projeto são: JavaScript (ES2022+) como linguagem única

para frontend e backend; Node.js (v22.20.0) como runtime; Express.js (v5.1.0) como framework

backend; React (v19.1.1) como framework frontend; Vite (v5.3.1) como build tool; Tailwind CSS

(v4.1.14) para estilização; Framer Motion para animações; Lucide React para ícones; React Router

DOM (v7.13.0) para roteamento; PostgreSQL via Supabase como banco de dados;

@supabase/supabase-js (v2.75.0) como cliente DB; JWT para autenticação; bcrypt para

criptografia; cors e express-rate-limit para segurança; Morgan para logging; swagger-jsdoc e

swagger-ui-express para documentação; Vitest para testes; Supertest para testes HTTP; React

Testing Library para testes frontend; e Git para controle de versão.

22 IMPLEMENTAÇÃO POR CAMADAS DO SISTEMA

22.1 Frontend

O frontend do REVIVE foi implementado como uma Single Page Application em React, com build e servidor de desenvolvimento pelo Vite. A estrutura está dividida em páginas, componentes, contextos, serviços, configuração e utilitários. As páginas principais são: Login, Dashboard, Analytics, Detalhes, Metas, Calendário, Conquistas, Relatórios, Perfil e Dicas.

O roteamento é feito com React Router. As rotas autenticadas ficam dentro do componente `ProtectedRoute`, que verifica se existe token e usuário no contexto de autenticação antes de liberar o acesso. O layout autenticado usa `AppShell`, com `Header`, `NavBar`, `ToastContainer`, `ConfirmModal`, `Alert` e `ErrorBoundary`. Essa composição permite reaproveitar a mesma estrutura visual em todas as telas internas.

O estado da aplicação é organizado em três contextos. O `AuthContext` controla login, cadastro, logout e verificação inicial do token. O `DataContext` carrega e altera vícios, registros, metas, recaídas e mensagem motivacional. O `UIContext` concentra tema, toasts, alertas, loading e modal de confirmação.

A comunicação com a API foi separada nos serviços `auth.service.js`, `vicios.service.js`, `registros.service.js`, `metas.service.js`, `recaidas.service.js`, `mensagens.service.js` e `api.js`. Essa separação facilita a manutenção, porque as telas não montam manualmente todas as requisições. A constante `API_BASE` é lida de `VITE_API_URL`, com fallback para `http://localhost:3000/api`.

O frontend também possui recursos complementares importantes, como exportação CSV na página de relatórios, exportação JSON no perfil, tema claro/escuro, gráficos e indicadores visuais, além de um Service Worker básico. Esse Service Worker faz cache do shell da aplicação e de assets locais, mas não implementa sincronização offline de dados. Por isso, ele deve ser descrito como suporte offline parcial.

Na revisão da implementação, foram encontrados dois ajustes necessários. O primeiro é o botão “Novo” da barra de navegação, que envia o usuário para `/novo-vicio`, uma rota que não está registrada no `App.jsx`. O segundo é a tentativa da `NavBar` de usar `vicioSelecionado`, enquanto o contexto trabalha com `selectedAddiction`. O cadastro de novo vício funciona pelo wizard do Dashboard, mas a navegação precisa ser alinhada para evitar caminho quebrado.

22.2 Backend

O backend foi implementado em Node.js com Express.js, concentrando a API REST no arquivo `index.js`. A aplicação carrega variáveis de ambiente com `dotenv`, configura CORS, interpreta JSON, registra logs com Morgan, aplica rate limiting e expõe a documentação Swagger em `/api/docs`.

As rotas de autenticação permitem cadastro e login. No cadastro, o sistema valida campos obrigatórios, exige senha com no mínimo seis caracteres, pelo menos uma letra maiúscula e pelo menos um caractere especial, verifica se o e-mail já existe e salva a senha com hash

bcrypt. No login, a API busca o usuário pelo e-mail, compara a senha com bcrypt e gera um JWT com validade de sete dias.

As rotas protegidas usam um middleware que extrai o token do cabeçalho `Authorization`, valida a assinatura com `JWT\_SECRET` e adiciona o identificador do usuário à requisição. A partir disso, as rotas de negócio filtram os dados pelo usuário autenticado quando necessário.

As funcionalidades de vícios permitem listar, criar, consultar detalhes e excluir. Na listagem e no detalhe, o backend calcula dias de abstinência, valor economizado e duração formatada. A exclusão de vício remove registros dependentes de `registros\_diarios`, `historico\_recaidas` e `metas` antes de apagar o vício. As recaídas podem ser registradas com ou sem reset do contador. Os registros diários armazenam humor, gatilhos, conquistas e observações. As metas podem ser criadas, listadas, concluídas e excluídas.

O backend apresenta uma base funcional consistente, mas existem limitações que precisam ser tratadas. A validação de entrada ainda é simples em vários pontos, e a função `sanitize()` apenas remove espaços nas extremidades. Também há uma rota de criação de meta que aceita `vicio\_id` sem verificar no próprio handler se o vício pertence ao usuário, dependendo mais da consistência do banco e da filtragem posterior. Para uma versão mais segura, essa verificação deveria ser adicionada.

22.3 Integração

A integração entre frontend e backend ocorre por requisições HTTP em JSON. O frontend chama os serviços, os serviços chamam `apiCall`, e `apiCall` adiciona o token quando necessário. O backend recebe as requisições, valida autenticação e dados básicos, consulta o Supabase e retorna respostas também em JSON.

No fluxo de autenticação, o usuário envia e-mail e senha pela tela de login. A API valida as credenciais e retorna o token. O frontend armazena esse token no `localStorage` e passa a usá-lo nas próximas chamadas. Ao recarregar a aplicação, o frontend tenta validar o token chamando a API. Esse comportamento funciona como persistência de sessão, mas ainda pode ser melhorado, pois a restauração atual não busca o perfil completo do usuário e pode exibir um usuário genérico.

Os testes executados confirmaram que a API responde ao health check, valida campos obrigatórios em login e cadastro, executa funções auxiliares e permite que partes do frontend sejam renderizadas com dados simulados. Porém, a integração completa com o Supabase ainda depende de testes mais amplos, principalmente para fluxos com banco real ou banco de teste controlado.

23 ORGANIZAÇÃO DO CÓDIGO E REPOSITÓRIO

O repositório do REVIVE segue uma estrutura monorepo com separação clara entre backend e

frontend. Na raiz estão o index.js (servidor Express), package.json, .env, vitest.config.js e start -

revive.cmd. A pasta docs/ contém openapi.js e coding -conventions.md . A pasta tests/ contém

routes.test.js e helpers.test.js. O subprojeto revive -painel/ contém o frontend React com package.json, .env, vite.config.js, vitest.config.js, public/ (com sw.js) e src/ (com main.jsx, App.jsx,

index.css, e pastas components/, cont exts/, pages/, services/, config/, utils/ e test/).

Padrões adotados: identificadores de código em inglês (camelCase para variáveis/funções,

PascalCase para componentes), textos de interface em português, componentes React em

arquivos .jsx, testes junto aos arquivos que testam ou em pasta tests/, e variáveis de ambiente

separadas para frontend e backend.

PARTE 06 – QUALIDADE, SEGURANÇA, AVALIAÇÃO E EXTENSÕES DO PROJETO

24 QUALIDADE DO SOFTWARE E TESTES

A estratégia de testes do REVIVE foi estruturada em camadas, usando Vitest como ferramenta principal. No backend, existem testes unitários para funções auxiliares e testes de integração para rotas Express com Supertest. No frontend, existem testes unitários de utilitários e testes de integração com React Testing Library para contextos e páginas.

Na revisão da Fase 02, foi executado o comando `npm run validate`. Esse comando roda os testes da API, os testes do painel e o build de produção do frontend. O resultado foi positivo: todos os testes passaram e o build foi gerado com sucesso.

Os testes unitários da API estão no arquivo `tests/unit/helpers.test.js`. Eles verificam a função `sanitize()`, a formatação de duração em dias e o cálculo de estatísticas de abstinência e economia. Esses testes se relacionam principalmente com os requisitos de acompanhamento de progresso e cálculo de economia, pois validam parte da regra que transforma dados do vício em informações exibidas ao usuário.

Os testes de integração da API estão em `tests/integration/routes.test.js`. Eles testam o endpoint `GET /api/health` e a validação de campos obrigatórios em `POST /api/auth/login` e `POST /api/auth/cadastro`. Esses testes se relacionam com o requisito de autenticação, pois verificam se a API rejeita requisições incompletas antes de acessar o banco.

No frontend, o teste `formatters.test.js` valida a função de cálculo de tempo decorrido exibida na interface. O teste de `AuthContext` verifica se, após o login simulado, o token e os dados do usuário são armazenados no estado e no `localStorage`. O teste de `DataContext` valida o tratamento de erro quando o carregamento de vícios falha. O teste da `ReportsPage` verifica se a página de relatórios renderiza dados agregados vindos do contexto.

Como evidência objetiva da execução, a validação local apresentou os seguintes resultados: 3 testes unitários da API aprovados, 3 testes de integração da API aprovados, 5 testes unitários do frontend aprovados, 3 testes de integração do frontend aprovados e build de produção concluído. No total, foram 14 testes automatizados aprovados, além da compilação do frontend.

Apesar do resultado positivo, a cobertura atual ainda é limitada. Os testes não executam um fluxo completo de usuário com banco real, como cadastro, login, criação de vício, registro diário, criação de meta e registro de recaída. Também não há teste automatizado para autorização entre usuários diferentes, nem teste específico para rate limiting, CORS, Service Worker ou proteção contra entradas maliciosas. Isso significa que os testes atuais ajudam a detectar regressões pequenas, mas ainda não comprovam a estabilidade completa do sistema.

Também não foi encontrado no repositório um roteiro formal de testes manuais. Pela natureza do sistema, os fluxos manuais mais importantes seriam: criar conta, fazer login, cadastrar um vício, registrar humor do dia, criar meta, registrar recaída sem resetar, registrar recaída resetando contador, consultar calendário, exportar relatório CSV, exportar dados JSON e testar logout. Esses fluxos devem ser documentados e executados na próxima etapa, com registro de resultado esperado e resultado obtido.

Em relação à qualidade interna, o projeto apresenta pontos positivos: separação entre serviços e telas no frontend, uso de contextos para estado compartilhado, `ErrorBoundary` no layout autenticado, tratamento de loading e toasts, documentação Swagger da API e pipeline de validação local. Por outro lado, a concentração de todo o backend em um único arquivo torna a manutenção mais difícil à medida que o projeto cresce. Uma evolução natural seria separar rotas, controllers, middlewares e serviços em arquivos próprios.

Com base na execução realizada, o estado atual da qualidade é satisfatório para uma entrega intermediária de faculdade, mas ainda não deve ser tratado como cobertura completa. A suíte confirma partes importantes do sistema, porém precisa evoluir para cobrir fluxos reais e regras de segurança antes de uma entrega final ou deploy público.

25 SEGURANÇA DA INFORMAÇÃO

A análise de segurança do REVIVE foi feita com base no código atual, nas dependências usadas e no schema Supabase informado. O sistema já possui algumas medidas importantes, mas também apresenta riscos que precisam ser tratados com prioridade proporcional ao impacto.

No armazenamento de senhas, o backend utiliza bcrypt com 10 salt rounds. Isso é uma medida adequada para não armazenar senhas em texto puro. No login, a comparação é feita com `bcrypt.compare()`, e o retorno de erro para credenciais inválidas é genérico, o que reduz o risco de enumeração de usuários.

A autenticação é feita com JWT. O token expira em sete dias e é validado pelo middleware antes das rotas protegidas. Esse modelo funciona para a aplicação atual, mas o frontend armazena o token no `localStorage`. O risco é médio, porque um ataque XSS poderia permitir roubo do token. Como mitigação, seria melhor avaliar cookies `HttpOnly` e `Secure`

em uma versão de produção, além de aplicar uma política de CSP e revisar pontos de entrada de HTML.

A autorização é tratada em várias rotas por meio do filtro ``usuario_id``. Por exemplo, a listagem e consulta de vícios filtram pelo usuário autenticado, e os registros/recaídas usam consultas relacionadas ao vício do usuário. Isso reduz risco de IDOR. Porém, a criação de meta aceita ``vicio_id`` opcional sem validar explicitamente se o vício pertence ao usuário. Esse risco é médio, porque pode gerar vínculo incorreto se o banco não impedir. A mitigação recomendada é validar o ``vicio_id`` antes de inserir a meta.

O acesso ao Supabase é feito pelo backend, e o frontend não usa diretamente as chaves do banco. Isso é positivo. Porém, o repositório não contém migrations nem políticas RLS. O schema informado mostra tabelas, chaves primárias, chaves estrangeiras e e-mail único, mas não mostra políticas de Row Level Security. Portanto, não é possível afirmar pelo material analisado que o isolamento também está garantido no banco. O risco é médio caso a chave usada tenha permissões amplas. A mitigação recomendada é documentar e versionar as políticas RLS ou garantir que apenas o backend tenha acesso privilegiado.

O CORS está configurado com lista de origens permitidas pela variável ``ALLOWED_ORIGINS``, com fallback para localhost em desenvolvimento. Isso reduz exposição indevida da API em navegadores. O rate limiting também está implementado: rotas de autenticação têm limite mais restrito, e as demais rotas ``/api`` possuem limite geral. Essa proteção reduz risco de força bruta e abuso, mas não substitui monitoramento ou bloqueio mais avançado em produção.

A validação de entrada existe em pontos importantes, como cadastro, login, criação de vício, criação de registro e criação de meta. Mesmo assim, ela ainda é básica. A função ``sanitize()`` apenas converte para string e aplica ``trim()``. Isso não deve ser descrito como proteção completa contra XSS ou injection. O risco é médio, principalmente porque campos como observações, gatilhos, conquistas e motivo de recaída aceitam texto livre. Como mitigação, recomenda-se validar tamanhos máximos, normalizar tipos esperados, escapar conteúdo na saída e usar biblioteca específica de sanitização quando necessário.

Quanto a SQL injection, o risco é baixo a médio. O projeto usa o SDK do Supabase, evitando montagem direta de SQL bruto na maior parte do código. Porém, a rota de mensagem motivacional usa o parâmetro ``tipo_vicio`` em uma expressão ``or(...)``. Ainda que o SDK reduza o risco de SQL injection tradicional, esse campo deveria ser validado contra valores esperados antes de ser usado na query.

O tratamento de erros foi parcialmente pensado. A função ``sendInternalServerError()`` oculta detalhes em produção, mas inclui ``detalhes`` quando ``NODE_ENV`` não é ``production``. Isso é útil durante desenvolvimento, mas deve ser conferido antes de deploy. Se o ambiente de produção for configurado incorretamente, detalhes internos podem aparecer para o usuário.

Não foi identificado risco alto comprovado no código analisado, considerando que o arquivo ``.env`` está ignorado pelo Git e que as chaves não aparecem versionadas. Os principais riscos atuais são médios e estão ligados ao token no ``localStorage``, à sanitização limitada,

à ausência de evidência local de RLS e a algumas validações de autorização que ainda podem ser reforçadas.

Como conclusão, o REVIVE possui uma base inicial de segurança coerente para o estágio do projeto: senha com hash, JWT, rotas protegidas, CORS e rate limiting. Mesmo assim, o sistema ainda não deve ser descrito como seguro de forma absoluta. Para uma entrega final mais madura, é necessário revisar permissões do Supabase, melhorar validações de entrada, corrigir a criação de metas com `vicio\_id`, criar `.env.example` com placeholders e avaliar uma estratégia mais segura para armazenamento do token no frontend.

26 EXTENSÕES E TECNOLOGIAS COMPLEMENTARES

O REVIVE utiliza as seguintes tecnologias complementares: Service Worker — Implementação

básica de Service Worker (public/sw.js) para suporte parcial a uso offline, permitindo que a aplicação funcione minimamente mesmo sem conexão à internet, o que é relev ante para o contexto do projeto pois permite que o usuário acesse seus dados em momentos de

indisponibilidade de rede. Swagger/OpenAPI — Documentação interativa automática da API REST

acessível em /api/docs, facilitando o desenvolvimento, testes e manutenç ão da API, criando uma

interface visual onde desenvolvedores podem testar endpoints diretamente no navegador.

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607

27 AVALIAÇÃO GERAL DO PROJETO INTEGRADOR

O desenvolvimento do REVIVE ao longo dos períodos representou uma jornada significativa de aprendizado técnico e amadurecimento profissional para toda a equipe. Mais do que entregar um sistema funcional, o projeto nos permitiu vivenciar na prática os desafios

reais de construir um produto de software do zero, passando por todas as etapas — desde a concepção da ideia até a implementação de uma aplicação completa com frontend, backend e banco de dados integrados.

Um dos principais aprendizados foi a importância do planejamento e da documentação. No

início, a tendência natural era partir direto para o código, mas rapidamente percebemos que definir regras de negócio claras, modelar os dados com cuidado e estruturar a arquitetura antes de implementar economizou um tempo enorme em retrabalho. A definição do pipeline de coleta de dados e a simulação com 20 participantes no semestre anterior, por exemplo, nos deu embasamento concreto para justificar decisões de design e priorização de funcionalidades, como o foco inicial em tabagismo e a integração de recursos voltados a gatilhos emocionais.

Entre as dificuldades enfrentadas, destacamos a curva de aprendizado com tecnologias que não dominávamos no início do projeto. A transição do protótipo mobile (React Native/Flutter) planejado originalmente para uma aplicação web com React foi uma decisão difícil, mas necessária diante da nossa experiência limitada em desenvolvimento mobile.

Essa escolha pragmática nos permitiu entregar um produto funcional e completo, ao invés de

um protótipo incompleto em uma tecnologia que ainda não dominávamos. Outro desafio relevante foi a integração entre frontend e backend — configurar corretamente o fluxo de autenticação JWT, o tratamento de CORS e a comunicação assíncrona entre as camadas exigiu bastante depuração e paciência.

As decisões mais relevantes do projeto incluem a adoção do Supabase como banco de dados,

que nos deu agilidade ao eliminar a necessidade de gerenciar infraestrutura de servidor de banco, e a escolha por um repositório monolítico (monorepo), que facilitou o desenvolvimento simultâneo de frontend e backend sem a complexidade de gerenciar múltiplos repositórios. A implementação do sistema de conquistas (badges) e do cálculo automático de economia financeira foram diferenciais que trouxeram valor real ao produto, tornando a experiência do usuário mais engajadora e motivadora.

Em termos de limitações, reconhecemos que o sistema ainda não possui notificações push em tempo real, funcionalidade que estava prevista no escopo original com Firebase Cloud Messaging. Além disso, a cobertura de testes, embora existente, pode ser ampliada para

cobrir cenários mais complexos de integração e fluxos de erro. O Service Worker implementado oferece suporte offline parcial, mas uma experiência offline completa exigiria estratégias mais avançadas de cache e sincronização de dados.

Como possibilidades de evolução futura, enxergamos grande potencial na implementação de

um modelo preditivo de recaída baseado nos dados de humor e gatilhos registrados pelos

Av. Universitária Km 3,5 - Cidade Universitária – Anápolis/GO, CEP 75083 -515

Secretaria Setorial – Bloco I

Fone: (62) 3310 6658 | FAX: (62) 3310 6607 usuários, conforme descrito no plano de evolução do semestre anterior. A integração com

APIs de saúde como Google Fit e Apple Health enriqueceria enormemente a análise de padrões comportamentais. Um módulo de comunidade anônima e moderada poderia endereçar o problema do isolamento social, e a migração para uma arquitetura de microsserviços com contêineres Docker permitiria escalar o sistema para atender um volume

maior de usuários. A gamificação avançada, com níveis e recompensas mais elaborados, também representa uma oportunidade de aumentar o engajamento.